EXPERIMENT 1

IMPLEMENTION OF CAESAR CIPHER

Aim:

To implement the Caesar Cipher substitution technique using Python.

Algorithm:

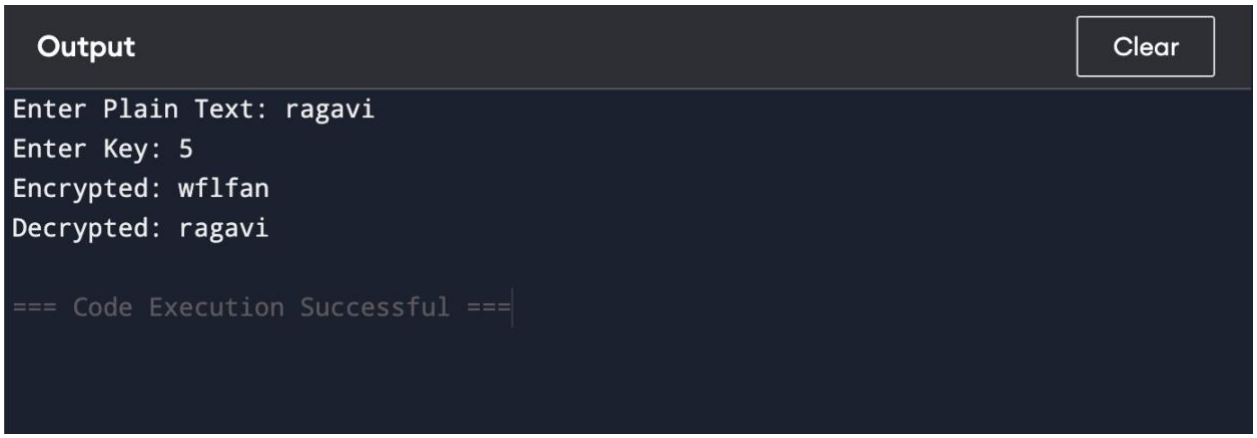
- Read the plaintext.
- Read the key (shift value).
- Shift each alphabet by the key value.
- Display the encrypted text.
- Decrypt using the same key.
- Display the original text.

Code:

```
    cipher = ""
    for ch in text:
        if ch.isupper():
            cipher += chr((ord(ch) - ord('A') + key) % 26 + ord('A'))
        elif ch.islower():
            cipher += chr((ord(ch) - ord('a') + key) % 26 + ord('a'))
        else:
            cipher += ch
    return cipher


def decrypt(cipher, key):
    text = ""
    for ch in cipher:
```

```
        if ch.isupper():
            text += chr((ord(ch) - ord('A') - key) % 26 + ord('A'))
        elif ch.islower():
            text += chr((ord(ch) - ord('a') - key) % 26 + ord('a'))
        else:
            text += ch
    return text


plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))
cipher = encrypt(plain, key)
print("Encrypted:", cipher)
print("Decrypted:", decrypt(cipher, key))
```

Output:



Result:

Thus, the Caesar Cipher substitution technique was implemented successfully using Python.

EXPERIMENT 2

IMPLEMENTION OF MONOALPHABETIC CIPHER

Aim:

To implement the Monoalphabetic Substitution Cipher using Python.

Algorithm:

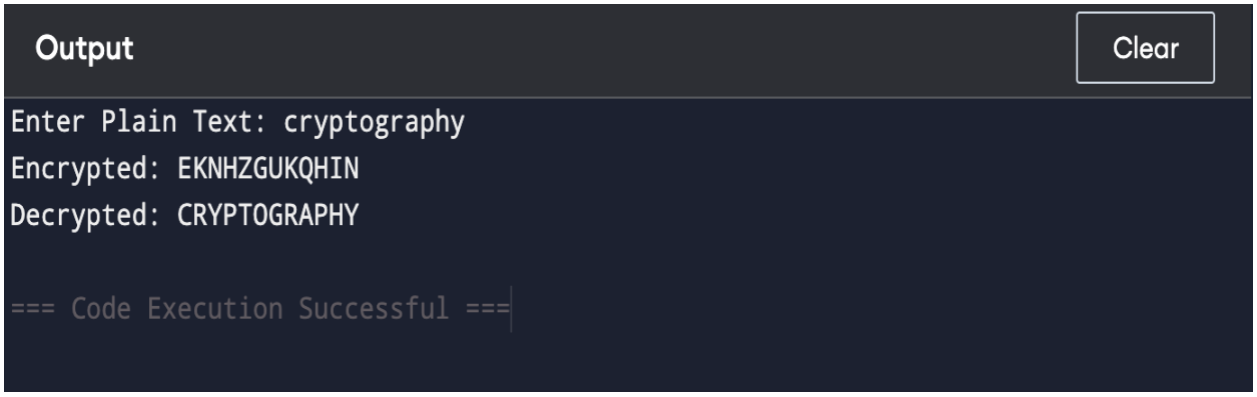
- Read the plaintext.
- Define the substitution key.
- Replace each letter using the key.
- Display the encrypted text.
- Perform decryption.
- Display the decrypted text.

Code:

```
import string

alphabet = string.ascii_uppercase

key = "QWERTYUIOPASDFGHJKLZXCVBNM"

plain = input("Enter Plain Text: ").upper()

cipher = ""

for ch in plain:


    if ch.isalpha():

        cipher += key[alphabet.index(ch)]

    else:

        cipher += ch


print("Encrypted:", cipher)

decrypted = ""
```

```
for ch in cipher:
    if ch.isalpha():
        decrypted += alphabet[key.index(ch)]
    else:
        decrypted += ch
print("Decrypted:", decrypted)
```

Output:



Result:

Thus, the Monoalphabetic Substitution Cipher was implemented successfully using Python

EXPERIMENT 3

IMPLEMENTION OF POLYALPHABETIC CIPHER

Aim:

To implement the Vigenère (Polyalphabetic) Cipher using Python.

Algorithm:

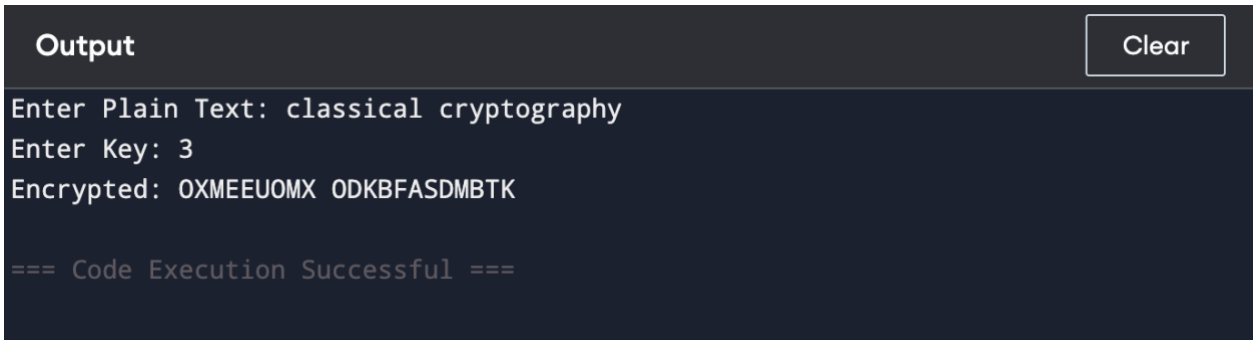
- Read the plaintext.
- Read the keyword.
- Repeat the keyword to match the text length.
- Encrypt the text.
- Decrypt the text.
- Display the results.

Code:

```
def generate_key(text, key):
    key = key.upper()
    text = text.upper()
    new_key = ""
    j = 0
    for ch in text:
        if ch.isalpha():
            new_key += key[j % len(key)]
            j += 1
        else:
            new_key += ch
    return new_key


def encrypt(text, key):
    key = generate_key(text, key)
    cipher = ""
```

```
    for i in range(len(text)):
        if text[i].isalpha():
            x = (ord(text[i].upper()) - 65 + ord(key[i]) - 65) % 26
            cipher += chr(x + 65)
        else:
            cipher += text[i]
    return cipher

plain = input("Enter Plain Text: ")
key = input("Enter Key: ")
print("Encrypted:", encrypt(plain, key))
```

Output:



Result:

Thus, the Vigenère (Polyalphabetic) Cipher was implemented successfully using Python.

EXPERIMENT 4

IMPLEMENTION OF HILL CIPHER

Aim:

To implement the Hill Cipher substitution technique using Python.

Algorithm:

- Read the plaintext.
- Read the key matrix.
- Convert letters into numbers.
- Perform matrix multiplication modulo 26.
- Convert numbers back into letters.
- Display the encrypted text.

Code:

```
key = [
    [6, 24, 1],
    [13, 16, 10],
    [20, 17, 15]
]

text = input("Enter Plain Text (3 letters): ").upper()

while len(text) < 3:
    text += "X"

text = text[:3]

P = [ord(ch) - 65 for ch in text]
C = []

for i in range(3):
```

```
    total = 0
    for j in range(3):
        total += key[i][j] * P[j]
    C.append(total % 26)
cipher = ""
for num in C:
    cipher += chr(num + 65)

print("Encrypted:", cipher)
```

Output:

Result:

Thus, the Hill Cipher substitution technique was implemented successfully using Python.

EXPERIMENT 5

IMPLEMENTION OF PLAYFAIR CIPHER

Aim:

To implement the Playfair Cipher substitution technique using Python.

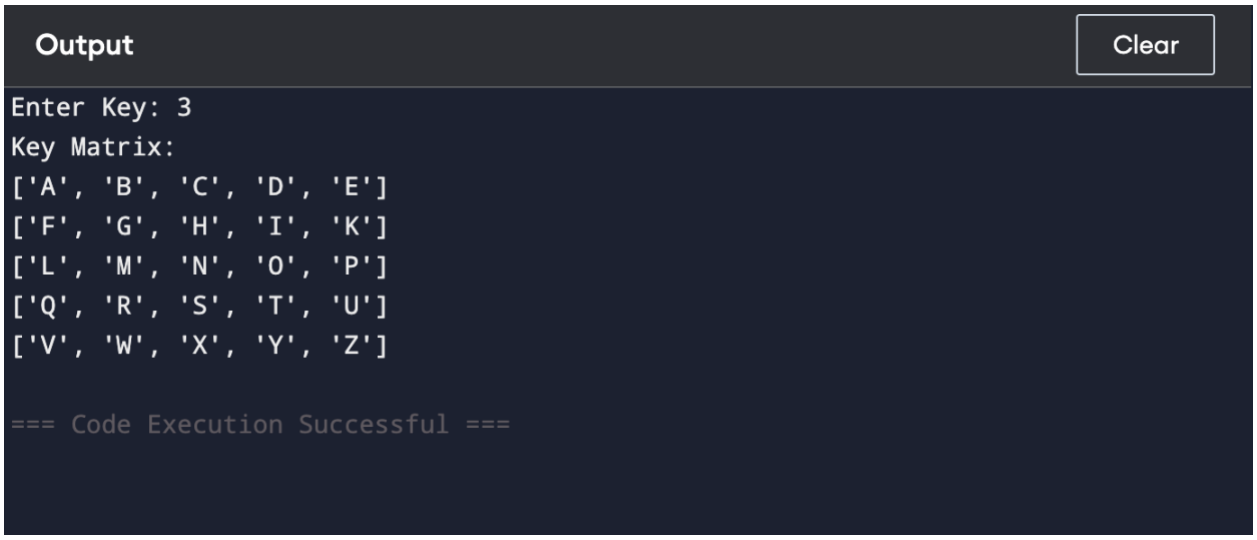
Algorithm:

- Read the plaintext and keyword.
- Generate the 5×5 key matrix.
- Split the text into pairs.
- Apply Playfair rules.
- Encrypt the text.
- Display the encrypted text.

Code:

```
def generate_key_table(key):
    key = key.upper().replace("J", "I")
    table = []
    for c in key:
        if c not in table and c.isalpha():
            table.append(c)
    for c in "ABCDEFGHIKLMNOPQRSTUVWXYZ":
        if c not in table:
            table.append(c)
    return [table[i:i+5] for i in range(0, 25, 5)]


key = input("Enter Key: ")
table = generate_key_table(key)


print("Key Matrix:")
for row in table:
    print(row)
```

Output:



Result:

Thus, the Playfair Cipher substitution technique was implemented successfully using Python

EXPERIMENT 6

IMPLEMENTION OF RAIL FENCE CIPHER

Aim:

To implement the Rail Fence transposition technique using Python.

Algorithm:

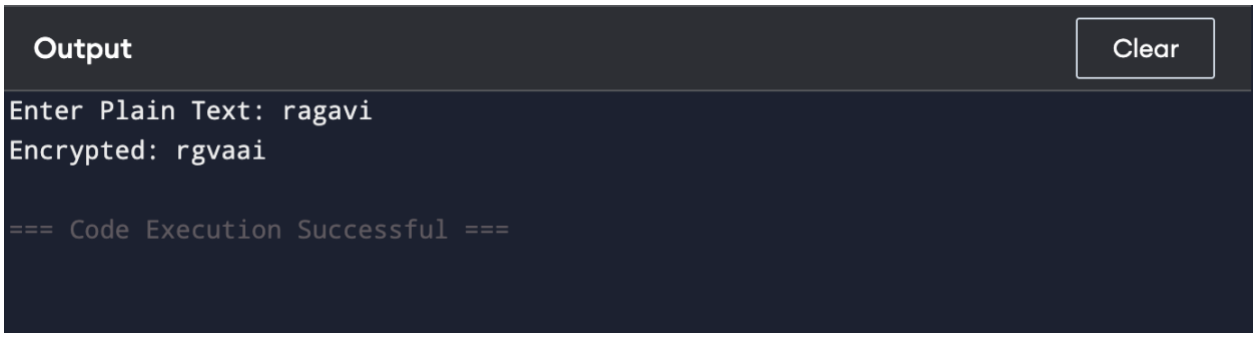
- Read the plaintext.

- Arrange the text in a zigzag pattern.
- Read row by row.
- Display the encrypted text.

Code:

```
def encrypt(text):
    rail1 = ""
    rail2 = ""

    for i in range(len(text)):
        if i % 2 == 0:
            rail1 += text[i]
        else:
            rail2 += text[i]

    return rail1 + rail2

plain = input("Enter Plain Text: ")
print("Encrypted:", encrypt(plain))
```

Output:



Result:

Thus, the Rail Fence transposition technique was implemented successfully using Python.

Experiment 7: Columnar Transposition Cipher

Aim

To implement the Columnar Transposition Cipher using Python.

Algorithm

Read the plaintext and keyword.

Arrange the text in rows.

Order columns according to the keyword.

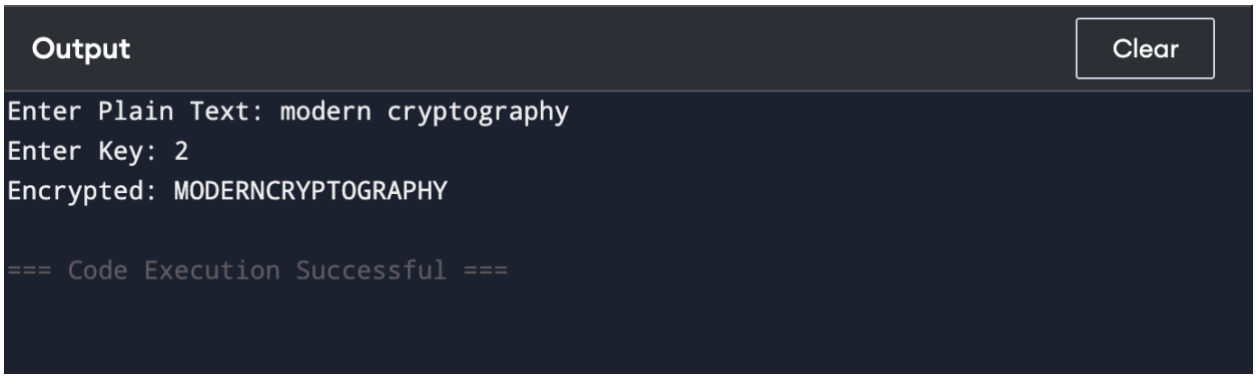
Read the columns in sorted order.

Display the encrypted text.

Code:

```
import math

text = input("Enter Plain Text: ").replace(" ", "").upper()

key = input("Enter Key: ").upper()

col = len(key) row = math.ceil(len(text) / col)

text += "X" * (row * col - len(text))

matrix = []

k = 0

for i in range(row):

matrix.append(list(text[k:k+col]))

k += col

order = sorted(list(enumerate(key)), key=lambda x: x[1])

cipher = ""

for index, _ in order:

for r in matrix:

 cipher += r[index]

print("Encrypted:", cipher)
```

Output:



Result:

Thus, the Columnar Transposition Cipher was implemented successfully using Python